



Modern Web Performance

Lecture 26
Mahhek Tahir



What is Web Performance

Performance is how fast and efficiently a web application responds to users.

Why it matters

- Faster apps → better user experience
- Slow apps → users leave
- Impacts scalability and cost



Type of Performance Issues

Client-side

- **Browser rendering** — Slow page display due to heavy HTML, CSS, or JavaScript rendering.
- **Network requests** — Delays caused while sending or receiving data over the internet.
- **Resource loading** — Slow loading of files like images, scripts, or stylesheets.

Server-side

- **Database queries** — Slow data retrieval or updates due to inefficient database operations.
- **CPU usage** — High processor consumption causing the system or application to slow down.
- **Request handling** — Delays in processing incoming user requests on the server.



Client-Side Performance



Techniques

- Minification (JS, CSS) - Remove Spaces Comments, Extra characters
 - **Terser** — JavaScript minifier. Terser
 - **UglifyJS** — JavaScript compression and minification tool. UglifyJS
- Compression: - Server compresses files before sending
 - Gzip - older - widely used
 - Brotli (modern, better, Smaller size than Gzip)
 - i. **Response coming back is compressed, file size reduced**
 - ii. **Server-side (most common)** — Enabled in servers like Nginx, Apache HTTP Server, or CDNs to compress files before sending to browser.
 - iii. **Node.js app** — Can also be implemented directly in a Node.js app using built-in **zlib** Brotli compression or middleware.
- Remove unnecessary HTML

Smaller files = faster downloads

Caching



Types

Browser Cache

- Stores static files
- Controlled using headers - Managed through HTTP cache headers like **Cache-Control** and **Expires**. (set by the **server**)

Server Cache

- Store API responses - Stores processed data or API responses on the server for faster future requests.

Example

First request → slow

Next request → instant

Problem

- Stale data: Cached content may become outdated if cache is not refreshed properly.

CDN (from “Exploit proximity”)



Content Delivery Network

Serve content from nearest location

Example

- Without CDN → far server → slow
- With CDN → nearby server → fast



HTTP/1

Old (HTTP/1)

Browsers had a **limit** (usually ~6 requests per domain)

So if a page had:

- 20 images
- CSS + JS files

Browser could NOT load all at once

Developers used **multiple domains** [img1.example.com](#)
[img2.example.com](#)



Modern (HTTP/2 & 3)

Modern (HTTP/2 & HTTP/3)

- Built-in parallel requests
- Multiplexing

Modern web can load multiple resources at once efficiently.



Multiplexing

Multiple requests at the same time on ONE connection, Browser doing multiple calls simultaneously on one connection

- No waiting
- No blocking

Requests do NOT wait for each other to finish

Multiple requests go at the same time

One slow request does NOT block others

Example:

- Browser requests:
 - `style.css`
 - `app.js`
 - `logo.png`

Parallel Loading

Browser can load:

- images
 - CSS
 - JS
- ALL together

Faster

More efficient

Earlier we needed multiple domains to load resources faster, but modern HTTP can handle multiple requests on a single connection using multiplexing.

Without multiplexing:

HTML → wait
CSS → wait
JS → wait
Image → wait

With multiplexing:

HTML + CSS + JS + Image
(all together)



Lazy Loading & Post-load

Techniques

- **Lazy loading images** — Images are loaded only when they become visible on the screen.
- **Load content when needed** — Data or components are fetched only when required by the user.
- **Infinite scroll** — More content automatically loads as the user scrolls down.

Benefit

- Faster initial load
- Better user experience



Lazy Loading Image

Images load **only when they are about to be visible**

Images below the screen **do NOT load initially**

They load when user scrolls or press arrow in image carousel



Load Content when needed

Data/content is loaded only when user requests it

“Load More” button

Clicking a tab

Opening a dropdown

View details

show image

Benefits

Less initial load

Faster UI

Efficient data usage



Infinite Scrolls

Examples

- Social media feeds
- Shopping websites

Benefits

- Smooth user experience
- No clicking “Next”
- Continuous browsing

Note

- Can be addictive
- Hard to reach footer sometimes



Server-Side Performance

Key Issues

- Too many DB queries
- Slow processing
- Blocking operations



Database Optimization

Techniques

- Use indexes
- Optimize queries
- Avoid unnecessary joins

Database is often the biggest bottleneck.

Eager loading

Load related data together early to avoid repeated queries and improve performance.



Caching on Server

Tools

- Redis
- Memcached

in-memory cache systems

Example

- Cache API response
- Avoid repeated DB queries



Cache api response

Example: Cache API Response

Without cache

1. User requests data
2. Server queries database
3. Sends response

Happens **every time**

With cache

1. User requests data
2. Server checks cache (Redis/Memcached)
 - If found → return instantly
 - If not → fetch from DB, store in cache



Load Balancing

Distribute traffic across multiple servers

Flow

Users → Load Balancer → Multiple Servers

Benefits

- **Prevent overload** — Stops one server from handling all requests alone.
- **Improve reliability** — If one server fails, others continue serving users.



Round Robbin

Requests go one by one to each server

- User 1 → Server A
- User 2 → Server B
- User 3 → Server C
- Then repeat

Equal distribution

Health Check

- Is server alive?
- Is it responding?



Least Connections

Send request to server with **fewest active users**

Better when load is uneven

IP Hash

Same user always goes to **same server**

Useful for login sessions

Weighted Distribution

Some servers are stronger

- Powerful server → more requests
- Weak server → fewer requests



Load Balancing Types

- Software (e.g. Nginx) - Runs on your own server
- Cloud (AWS, Azure) - Provided by cloud platforms - Amazon Web Services (AWS) - Microsoft Azure - Managed for you - Auto scaling + high availability
- DNS-based - DNS gives different IPs to users

No single point of failure - if one fail request goes to other

DNS



DNS-based load balancing means the DNS server returns different server IP addresses to different users to spread traffic.

Example:

example.com → 1.1.1.1

example.com → 1.1.1.2

example.com → 1.1.1.3

Flow:

1. User opens website
2. DNS resolves domain name
3. DNS gives one of multiple server IPs
4. User connects to that server

Benefit:

- Traffic gets distributed across servers
- Reduces overload on a single server



Measuring Performance

Why? To know:

- How fast your app is
- How it behaves under load
- Where bottlenecks are

Server-side

- Apache JMeter
 - a. Simulates many users
 - b. Tests server under heavy load
 - c. Finds breaking point

Client-side

- Chrome DevTools
- Lighthouse

You can't improve what you don't measure. Measure and optimize

Comparison



Tool	Type	What it Tests	Use Case
Apache JMeter	Server-side	Load, stress, concurrent users	"Can my server handle 1000 users?"
Chrome DevTools	Client-side	Network requests, timing, rendering	"Why is my page slow?"
Google Lighthouse	Client-side	Performance score + suggestions	"How to improve performance?"

References



Web Performance (Core Concepts)

- <https://web.dev/performance>

Core Web Vitals (Modern Metrics)

- <https://web.dev/explore/learn-core-web-vitals>

Caching

- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cache-Control>

CDN & Content Delivery

- <https://web.dev/articles/content-delivery-networks>
- <https://docs.cloud.google.com/cdn/docs/best-practices>

Performance Measurement

- <https://web.dev/articles/vitals-field-measurement-best-practices>

General Understanding

- https://en.wikipedia.org/wiki/Web_performance